



Smart glasses that listen for machine faults.

A self-funded hardware pivot that became a benchmarked audio ML system.

WHY I BUILT IT

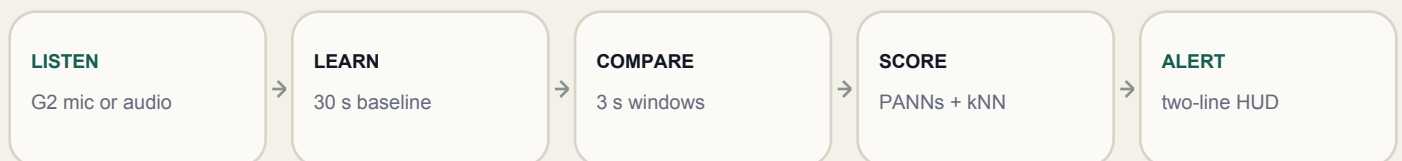
About six months ago, my boxing coach came to the gym wearing Ray-Ban Meta glasses. Seeing someone I knew use smart glasses made the technology feel real. I wanted to build for that future, so I completed coding bounties and programming work for my dad, saved the money, and bought Even Realities G2 glasses myself. EarSight grew from one question: could smart glasses give a technician another way to notice a machine problem before it becomes obvious?

THE IDEA



Learn a healthy sound. Flag the change.
Explain it in two lines.

HOW EARSIGHT WORKS



Current implementation: local scoring engine, no cloud inference required



Bearing-like anomaly | 98%

high-band energy rising | tap to log

TWO-LINE G2 HUD CONTRACT

WHAT MAKES IT DIFFERENT

Most monitoring systems attach a sensor to one machine. EarSight explores a different model: the microphone moves with the technician, while the display stays hands-free. The same pair of glasses can move between machines, surface an anomaly, and save evidence for later review.

BUILT WITH

Python / PyTorch
TypeScript / Even Hub
PANNs / kNN / ONNX



What I proved, and what I still need to prove.

The benchmark is proven. The real G2 microphone is the next gate.

SCALE

9,755

fan and pump recordings in the full MIMII evaluation

FAN AUC

0.693

DCASE reference: 0.658 | +3.5 points

PUMP AUC

0.880

DCASE reference: 0.729 | +15.1 points

DCASE is an international research challenge for detecting and classifying sounds. Its 2020 Task 2 autoencoder is the official reference model for machine anomaly detection. EarSight was fit only on healthy machine clips, then evaluated on held-out healthy and faulty clips. AUC is a ranking metric where 0.5 is chance and 1.0 is perfect.

THE FAILURE I DID NOT HIDE

A fan-versus-pump classifier reached 1.000 training accuracy, then dropped to 0.577 on unseen machine IDs. I removed it from the live path instead of presenting the training score. The anomaly detector uses a separate per-machine design and was unaffected.

HOW I BUILT IT

This was my first project using Claude Code with Fable 5. I divided the build into about 22 independently testable tasks, wrote the acceptance criteria, and checked the results. The lesson was simple: faster code generation makes verification more important, not less.

VALIDATED TODAY

- Full MIMII fan and pump evaluation
- Deterministic double-pass across all reported metrics
- Live streaming, evidence, and labeling engine
- Even Hub simulator and real-engine integration

NEXT G2 HARDWARE GATE

- Confirm continuous PCM capture from the real glasses
- Verify sample rate with a known 1 kHz tone
- Re-record the pump test through the G2 microphone
- Measure background operation and battery cost

The part I am proudest of is that I was willing to find out where it did not work.

Full 63-page build transcript included in the application folder | Source: engine/REPORT_FULL.md